

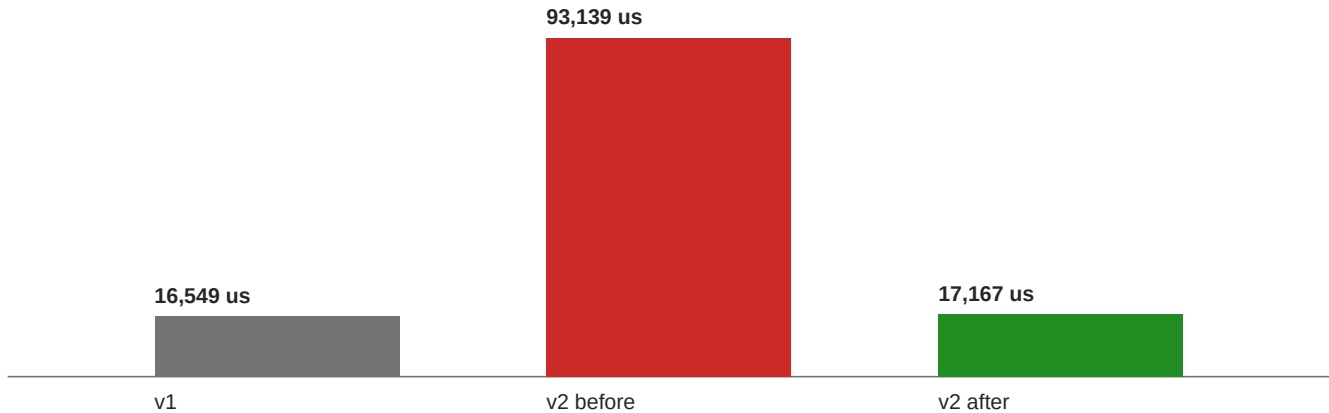
cuda.compute v2 (HostJIT) - Performance Report

NVIDIA RTX 6000 Ada • 2026-05-30 • fix commit 5697955ac0 • Elements = 2^{28}

Summary

Benchmarking the new HostJIT (v2) backend against the legacy NVRTC/LTO backend (v1) surfaced one severe regression: heavy compute operators ran ~5.6x slower on v2. Root cause was a missing loop-unroll/SROA step in the HostJIT optimization pipeline that left user-operator local arrays in local memory. A pipeline fix restores parity with v1 on that kernel and is neutral elsewhere.

Headline: transform/heavy (operator with a 64-elem local array)



v1 16,549 us vs v2 before 93,139 us (0.18x of v1) vs v2 after 17,167 us (0.96x). The fix is a 5.4x speedup on this kernel, restoring v1 parity. Geomean across the full suite improved from ~9% slower than v1 to ~3% slower (nearly all of the gain from this kernel); every other kernel is unchanged within noise.

Identified problem

cuobjdump showed the heavy kernel used a 256-byte local-memory stack frame with ~670 local load/store (LDL/STL) instructions on v2, vs zero on v1 (where the operator's uint32[64] array lives in registers). The optimized IR confirmed the cause: HostJIT's default O2 pipeline only PARTIALLY unrolled the operator's fixed-trip-count loops, so the backing alloca kept a dynamic index and SROA could not promote it. ptxas does this promotion on the v1/LTO path; the LLVM-NVPTX path needs full-unroll-then-SROA at the IR level.

Resolution

In `c/parallel.v2/src/hostjit/compiler.cpp`, raise LLVM's loop-unroll thresholds so the operator's small constant-trip loops fully unroll, then re-run the optimization pipeline so SROA promotes the now-constant-indexed arrays to registers. (Also adds a `CCCL_HOSTJIT_DUMP_DIR` debug hook for IR/PTX dumps.)

Remaining v2 gaps (separate root causes, not this fix)



These were unchanged by the fix. `segmented_reduce/variable_sum` (~1.8x slower) is the largest and is likely the v2 segmented-reduce dispatch/signature path, not codegen. `segmented_sort/keys [power]` slipped ~4% AFTER the fix because the unroll threshold is currently a process-wide setting - an argument for refining to a scoped, single appended SROA pass (which also halves the JIT compile cost).